

Algoritmos y estructuras de datos

Árboles

Recursividad

Es cuando una función llama a la misma función que se está definiendo. Una función se llama a sí misma.

Por ejemplo, tomando el factorial del número 5 ($5 \times 4 \times 3 \times 2 \times 1$), que es lo mismo que $5 \times 4!$. Entonces $n!$ es igual a $n \times (n-1)!$. En esta definición estamos definiendo al factorial en términos de sí mismo, es decir, es una definición recursiva.

Cuando utilizamos recursividad dejamos cálculos pendientes hasta que hallemos un valor que me permita resolver todo el cálculo, un valor no recursivo. Siempre tiene que haber un valor por el cual se resuelva directamente, que no esté definido en términos de sí mismo.

Para recorrer un árbol si o si debo utilizar la recursividad o simularla a través del uso de pilas.

En términos de código, vamos a hacer una función factorial que reciba un número.

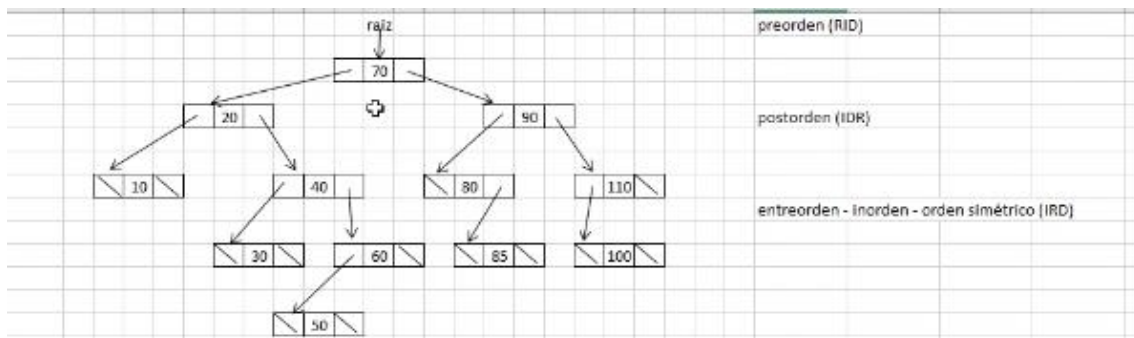
Lo primero que hacemos es preguntar si n (número) es igual a 0. En caso de que lo sea, entonces devolvemos 1, pues el factorial de 0 es 1. Si no es 0, va a ser mayor, y devolvemos $n \times \text{factorial}(n-1)$. De esta forma estamos haciendo una función recursiva que devuelve el factorial de un número.

```
int factorial(int n)
{
    if(n==0)
        return 1;
    else
        return n*factorial(n-1);
}
```

Árboles binarios

En un árbol, el puntero de acceso es un puntero al nodo raíz. Es una estructura recursiva ya que tenemos un puntero de acceso al nodo raíz, que tiene un árbol por izquierda y por derecha. Cada nodo tiene un árbol a izquierda y a derecha.

No es una estructura secuencial ya que no se puede definir con claridad cuáles son los elementos que le siguen a los nodos.



Para recorrer un árbol hay 3 formas:

Preorden: primero accedo a la raíz, después al subárbol izquierdo y después al derecho. Después, en cada subárbol realizo el mismo recorrido.

En este caso el preorden queda 70, 20, 10, 40, 30, 60, 50, 90, 80, 85, 110, 100

Postorden: primero accedo al subárbol izquierdo, después al de la derecha y después a la raíz. Después, en cada subárbol realizo el mismo recorrido.

En este caso el postorden queda 10, 30, 50, 60, 40, 20, 85, 80, 100, 90, 70

Entreorden – inorden – orden simétrico: primero accedo al subárbol izquierdo, después a la raíz y por último el subárbol derecho. Después, en cada subárbol realizo el mismo recorrido.

En este caso el entreorden queda 10, 20, 30, 40, 50, 60, 70, 80, 85, 90, 100, 110

El árbol está armado de la forma entreorden, por eso queda ordenado de menor a mayor. Este es el orden que nos interesa en la materia. También se lo conoce como ABB o Árbol Binario de Búsqueda. Todos los valores que están a la izquierda de la raíz son menores y los que están a la derecha son mayores. Por eso me permite hacer una búsqueda binaria de forma más eficiente que con cualquier otra estructura.

Definimos un nodo que tiene 3 elementos, int info (o de otro tipo de dato), Nodo*izq y Nodo*der.

Función ListarIn

Va a recibir la raíz del árbol. Lo que hace el recorrido es listar el árbol izquierdo, imprimir la raíz y después listar el árbol de la derecha. Para hacer eso primero tenemos que mostrar todo el árbol izquierdo en inorden. Primero preguntamos si el árbol es distinto de NULL. Si lo es, entonces llamo a listarIn dentro de la propia función y le mando a esta función la raíz->izq. Una vez que termina de ejecutarse esta función voy a mostrar la raíz->info con un cout. Por último, llamo a listarIn y le mando raíz->der.

```

void listarIn(Nodo*raiz)
{
    if (raiz!=NULL)
    {
        listarIn(raiz->izq);
        cout<<raiz->info<<endl;
        listarIn(raiz->der);
    }
}

```

Función ListarPre

Acá primero pregunto si la raíz es distinta de NULL. En ese caso primero muestro la raíz->info, después llamo a listarPre y le mando el árbol de la izquierda y después lo mismo pero con el árbol de la derecha.

```

void listarPre(Nodo*raiz)
{
    if (raiz!=NULL)
    {
        cout<<raiz->info<<endl;
        listarPre(raiz->izq);
        listarPre(raiz->der);
    }
}

```

Función ListarPos

Acá primero pregunto si la raíz es distinta de NULL. En ese caso primero llamo a listarPos y le mando el árbol izquierdo, después el derecho y después la raíz.

FOTO

Buscar en un árbol

Para buscar en un árbol no necesitamos usar recursividad, porque no voy a recorrer todo el árbol. Sólo me voy a fijar si está a la izquierda o a la derecha recorriendo las ramas.

Vamos a hacer una función Nodo* buscar que va a recibir un número y me va a devolver en donde se encuentra dicho número en el árbol. Recibe el Nodo*raíz y el dato a buscar.

Primero defino una variable de recorrido r inicializada en raíz. Hago un ciclo while mientras r sea distinto de NULL y mientras r->info es distinto a n, o sea, si no termine de recorrer la rama y no estoy parado en el elemento que busco. En este caso me muevo al siguiente. Para eso pregunto si n es menor a r->info, entonces el siguiente va a ser el de la izquierda, o sea r=r->izq. Si no, va a ser el de la derecha.

Por último devuelvo r, que es el puntero al nodo que tiene el elemento que busco.

```

    Nodo*buscar(Nodo*raiz,int n)
    {
        Nodo*r=raiz;
        while(r!=NULL && r->info!=n)
        {
            if(n<r->info)
                r=r->izq;
            else
                r=r->der;
        }
        return r;
    }
}

```

Generar un árbol

Primero vamos a hacer una función insertar que va a recibir el Nodo*raíz y un número que es el que vamos a insertar.

Vamos a insertar un nodo hoja que tiene los dos punteros en NULL, o sea no tiene subárboles. Siempre le vamos a ir agregando hojas. La raíz no se va a modificar nunca salvo el caso en que la raíz esté en NULL. Entonces la función insertar va a recibir el Nodo*raíz por referencia porque no se que caso se puede dar.

Defino una variable para mi nuevo nodo Nodo*n y pido memoria con new Nodo. A n->info le asigno número y a n->izq y a n->der le asigno NULL, ya que es una hoja.

Luego pregunto si el árbol está vacío, o sea si raíz es NULL. En dicho caso lo que voy a hacer es asignarle a raíz el nodo n que agregué.

Si el árbol no está vacío, tengo que encontrar quién va a ser el nodo del árbol que apunte a n. Tendré que ir recorriendo la rama donde tengo que enganchar al nodo. Entonces voy a hacer el mismo recorrido de la función buscar pero solo mientras r sea distinto de NULL. Lo que voy a hacer es declarar una variable ant donde voy a ir guardando el anterior r antes de pasar a la izquierda o a la derecha. De esta forma ant me va a quedar apuntando al elemento que quiero que apunte a la hoja que quiero agregar.

Una vez que salgo del while pregunto si el número es menor a ant->info. Esto quiere decir que lo tengo que poner a la izquierda de ant. O sea: ant->izq=n.

Si el número no es menor que ant, entonces lo engancho a la derecha de ant, o sea ant->der=n.

```

3 {
    Nodo*n=new Nodo;
    n->info=nro;
    n->izq=n->der=NULL;
    if(raiz==NULL)
        raiz=n;
    else
    {
        Nodo*ant,*r=raiz;
        while(r!=NULL)
        {
            ant=r;
            if(n<r->info)
                r=r->izq;
            else
                r=r->der;
        }
    }
}

}
if(nro<ant->info)
    ant->izq=n;
else
    ant->der=n;
}

```

En el main tenemos que definir el Nodo*raíz en NULL. Después ingresamos el dato y luego llamamos a la función insertar. El árbol va a crearse depende de cómo ingrese los datos, pero el orden siempre va a tener el mismo orden cuando lo listemos.

```

int main()
{
    Nodo*raizArbol=NULL;
    int num;
    cin>>num;
    while (num!=0)
    {
        insertar(raizArbol,num);
        cin>>num;
    }
    return 0;
}

```